

PROJECT AND TEAM INFORMATION

Project Title

ExamSync – Smart Examination Coordination Portal

Student / Team Information

<i>Team Name:</i> <i>Team Id :</i>	<i>Code Quartet</i> <i>DSCPP-III-2026-T079</i>
Team member 1 (Team Lead) <i>(Last Name, Name, Student ID, Email):</i>	<i>Sharma, Shreeya - 2510011488</i> <i>shreeyasharma033@gmail.com</i> 
Team member 2 <i>(Last Name, Name, Student ID, Email):</i>	<i>Bahukhandi, Inakshi - 2512520002</i> <i>inakshi.8734@gmail.com</i> 
Team member 3 <i>(Name, Student ID, Email):</i>	<i>Rincy - 2510010306</i> <i>rincybadhawa.1998@gmail.com</i> 
Team member 4 <i>(Last Name, Name, Student ID, Email):</i>	<i>Jha, Saumya - 2510011508</i> <i>saumya.jha251@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Examination related works are handled through seating-plan documents and common lists. The seating arrangement along with other room details are shared in from of lists, thus the students have to search for their roll number to find out their respective room and seat.

For teachers, the examination duties are generally intimated through a common duty list. In case of any UFM (Unfair Means) case during an examination, the details are manually noted down on paper for further checking by the concerned authorities.

Although the current system handles all these works, finding out individual information from common lists might take time and be tedious. The goal of our project is to make the process easier, more organized and personalized by providing the user with all the information that concerns him/her through a common portal.

State of the Art / Current solution

Currently, the students need to refer to the common seating list to find their seating details. It can take a long time to get the required information including building, floor, classroom and seat numbers which could also be confusing.

For the administrative personnel, the process starts with getting the student and room information and working on a tentative seating plan.

Any change in student's details, the room or its availability, or the exam schedules would require redoing the seating arrangement.

While the current system is good at providing the required seating information, it can be observed that as the number of students and rooms increase the process of allocating them becomes tedious and requires multiple checks and balances.

This indicates the need of having an intelligent seating allocation system that can manage the seating arrangements and their updates in a much more systematic and automated manner while eliminating the human errors.

Project Goals and Milestones

1. Project Goals

- *To design an intelligent system for allocating seats that decreases the scope of error.*
- *Allow students to fetch the details of their allotted seat easily by inputting their university roll number.*
- *Display building, floor, room and seat information through the portal.*
- *Provide teacher specific invigilation duties.*
- *Manage UFM records digitally.*
- *Apply DSA concepts to seating allocation and data management.*

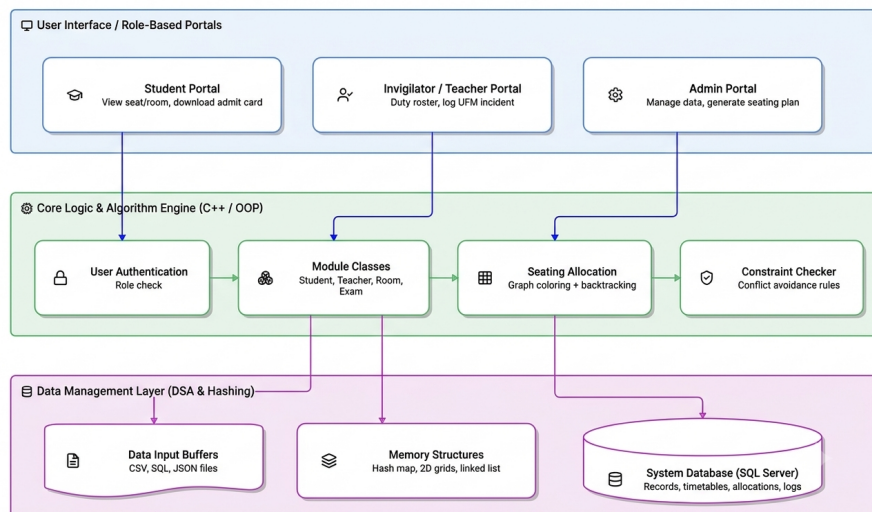
2. Milestones

- *Data collection, Database and detailed analysis of the collected data.*
- *Seating allocation algorithm.*
- *Creating separate modules for students and teachers.*
- *Admin module and UFM management.*
- *Integration and testing.*

Project Approach

1. *Requirement Analysis*
 - The data consists of student details, examination details, rooms, available seats, and teacher duties.
 - The modules and portal working are decided based on requirements.
2. *Implementation of OOPs*
 - The system's logic related to Object-Oriented Programming has been implemented in C++.
 - Classes have been created for Student, Teacher, Room, Examination, and Seating Arrangement.
 - Encapsulation maintains data and functions together; constructors initialize the objects; inheritance and polymorphism share properties; operator overloading makes operations easier.
3. *Data Management using DSA*
 - Arrays are used to store static records whereas pointers or linked lists are used to store dynamic data.
 - Hashing technique is used to fetch the desired record quickly.
 - Searching and sorting algorithms find and order records efficiently.
4. *Generating the Smart Seating Plan*
 - The admin enters student, examination, room, and seat details.
 - The system assigns seats according to DSA algorithms to prevent conflicts.
5. *Student Module*
 - The students log in to view examination information.
 - Searching and hashing retrieve examination and seating information.
 - The portal provides an option to access or download the admit card.
6. *Teacher Module*
 - The teachers log in to view their assigned duty (Invigilator / Flying Squad).
 - Searching finds the duty assigned to a teacher.
 - The authorized faculty records the UFM information.
7. *Admin Module*
 - The admin manages student, teacher, examination, room, and seating records using DSA and hashing techniques.
 - Searching and sorting find records; the admin generates seating arrangements.

System Architecture



Project Outcome / Deliverables

- 1- A smart examination portal will make the examination process more efficient.*
- 2- Smart and efficient seating allocation module which makes the allocation process easier and reduce errors.*
- 3- Information about the allocated seats of each student will be easily accessible and will prevent confusions.*
- 4- Admin dashboard.*
- 5- Teacher duty dashboard.*
- 6- Unfair Means cases, which are currently handled manually will be managed through the portal.*
- 7- Seating plan generation.*
- 8- Data storage.*
- 9- Documentation and testing.*

Assumptions

Access: The administrators have access to all the student and teacher information.
Schedules: Scheduling of exams is done prior to finalizing the seating plans
Capacity: Seating capacity and room sizes are pre-set.
Security: Restriction of editing exam information is given only to authorized users.
UFM Records: Only authorized faculty or admins can report unfair means (cheating) cases.

References

<https://www.geeksforgeeks.org/>
<https://www.w3schools.com/>
<https://www.programiz.com/>
Visual Studio Code